



PARALELIZAÇÃO DE UM ALGORITMO GENÉTICO

Pedro H. F. Fleck Leonardo R. dos Santos Miguel D.
F. Guerra

Trabalho Prático - Programação Paralela — 29 de setembro de 2026

1. O Problema
2. Paralelismo de Tarefas
3. Paralelismo de Dados
4. Desempenho e Escalabilidade
5. Análise Física (Intel VTune)

- **Objetivo:** Otimização e busca baseada em evolução biológica.
- **Análise de Gargalo Computacional:**
 - Inicialização da população ocorre apenas uma vez (custo residual).
 - O laço geracional principal consome o tempo da aplicação.
 - Avaliação de *fitness* e reprodução são processos matematicamente intensivos.
- **Estratégia:** Paralelizar exclusivamente as regiões críticas para evitar *overhead* de criação de threads em etapas únicas.

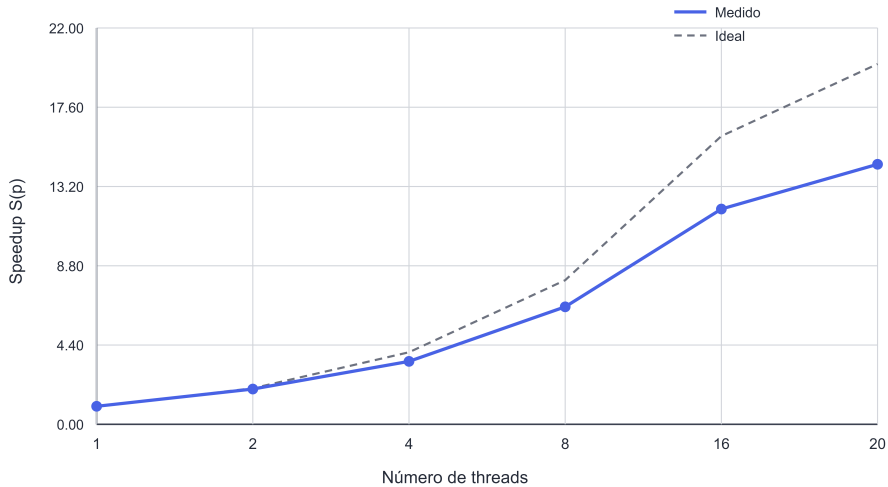
- **Divisão Estática:** `#pragma omp parallel for schedule(static)`.
- Todos os indivíduos possuem a mesma dimensão em bits, resultando em custo computacional idêntico por iteração. O escalonamento estático elimina custos de rebalanceamento.
- **Estado Isolado (Zero Locks):**
 - Zonas críticas de bloqueio causam contenção e degradam o desempenho.
 - O Gerador de Números Aleatórios (RNG) foi tornado privado.
 - Cada thread instancia e opera seu próprio ponteiro de estado (`rng_state`), gerando indivíduos filhos em posições disjuntas de memória (sem sobreposição).

```
1  /* Each iteration uses its own RNG state and writes one disjoint child. */
2  #pragma omp parallel for schedule(static)
3  for (size_t i = 0; i < population_size; i++) {
4      uint64_t state = rng_seed_for_individual(base_seed, generation, i);
5      const size_t first_parent = tournament_select(fitness, population_size, &state);
6      const size_t second_parent = tournament_select(fitness, population_size, &state);
7      const size_t cut = 1 + rng_index(&state, bits_per_individual - 1);
8      const uint8_t *parent1 = population + first_parent * bits_per_individual;
9      const uint8_t *parent2 = population + second_parent * bits_per_individual;
10     uint8_t *child = next_population + i * bits_per_individual;
11
12     for (size_t bit = 0; bit < bits_per_individual; bit++) {
13         child[bit] = bit < cut ? parent1[bit] : parent2[bit];
14         if (rng_unit(&state) < mutation_rate) {
15             child[bit] ^= 1u;
16         }
17     }
18 }
```

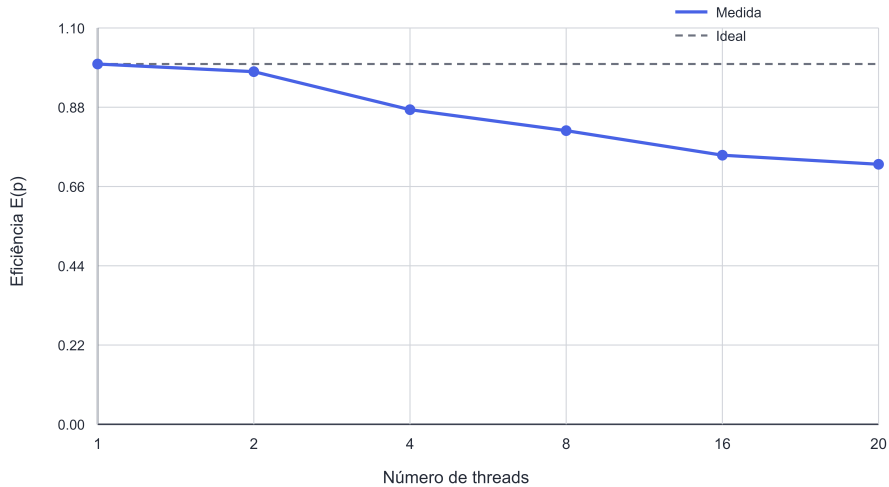
- **Alvo de Otimização:** Função de avaliação (`individual_evaluate`).
- Operação composta por um laço linear que soma valores em memória de forma totalmente independente, ideal para unidades vetoriais largas.
- Uso de `#pragma omp simd reduction(+:fitness)` para acumular o resultado sem conflitos de registradores.
- **Inviabilidade no código de reprodução:** O laço de *crossover* apresenta dependência de dados carregada pelo laço (*Loop-Carried Dependency* no RNG) e divergência por desvios de *branch* (taxa de mutação).

```
1 static int individual_evaluate(const uint8_t *individual,
2                               size_t bits_per_individual) {
3     int fitness = 0;
4
5     // Inclusion of SIMD, linear iteration of data
6     #pragma omp simd reduction(+:fitness)
7     for (size_t bit = 0; bit < bits_per_individual; bit++) {
8         fitness += individual[bit];
9     }
10
11     return fitness;
12 }
```

Speedup de escalabilidade - conjunto grande



Eficiência - conjunto grande



Testes realizados no nó hype1 (Intel Xeon E5-2650 v3 @ 2.30GHz, 20 núcleos).

- **Carga Avaliada:** População 4000, 2000 bits, 100 gerações.
- **Speedup e Eficiência:**
 - Escalabilidade contínua até o limite de núcleos físicos ($p = 20$).
 - Speedup de 14.43x (em relação à versão paralela com 1 thread).
 - Aceleração T_{seq}/T_{par} de 15.85x em comparação com o código puramente sequencial.
 - Manutenção de 72.2% de eficiência sob carga máxima.

Coleta com 20 threads:

- **Eficiência da CPU:**

- CPI Rate: 0.395.
- Elapsed Time: 1.064s.
- Dispatch médio superior a 2 instruções por ciclo, evidência da vetorização dos registradores.

- **Distribuição de Carga:**

- Effective Physical Core Utilization: 89.0%.
- Estratégia de *zero-locks* e escalonamento estático assegura atividade simultânea efetiva em quase 18 dos 20 núcleos.

- **Uso de Memória:**

- Memory Bound: 2.9% (apenas 5.6 GB/s de 124 GB/s).
- O algoritmo é restrito por capacidade de cálculo (FPU) e não por limites de transferência da DRAM.

Pedro H. F. Fleck Leonardo R. dos Santos Miguel D. F. Guerra
Instituto de Informática — UFRGS

